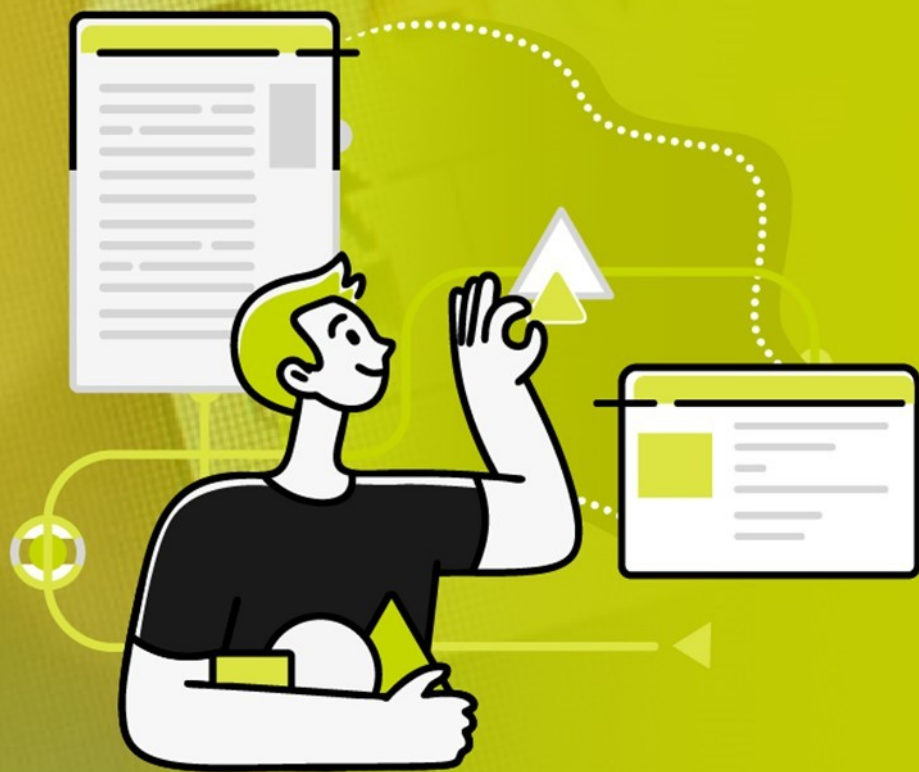




Carrera **Desarrollo y Diseño Web**

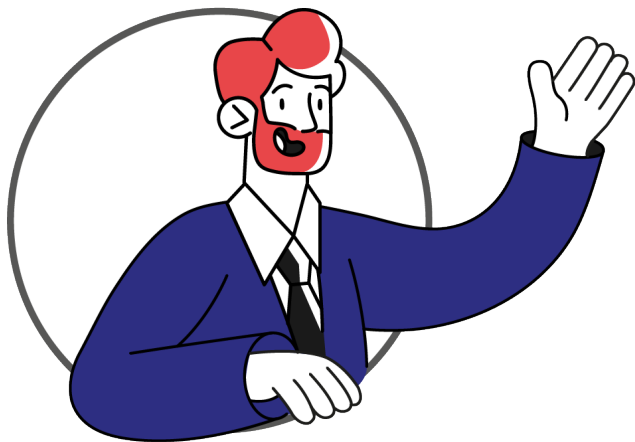
Experiencia 2 – Semana 4

Manipulando el DOM

Índice

Introducción a la semana.....	3
Resultado de aprendizaje.....	4
Conceptos relevantes.....	6
Preguntas activadoras.....	6
Actividad.....	7
¿Qué es el DOM?.....	8
Árbol de objetos.....	9
Atributos.....	11
Selección de elementos en el DOM.....	12
Seleccionando elementos con Javascript.....	13
Modificar elementos.....	18
Operaciones sobre conjuntos de elementos.....	22
Cierre de la semana.....	23
Referencias.....	24
Apuntes.....	25

Introducción a la semana



El Document Object Model (DOM) en JavaScript es una herramienta fundamental para interactuar dinámicamente con el contenido de una página web. En esencia, representa la estructura del documento como un árbol de nodos, donde cada nodo corresponde a un elemento, atributo o fragmento de texto en el documento HTML. Aprender a utilizar el DOM implica comprender cómo JavaScript puede

acceder, modificar y manipular estos nodos, permitiendo así la creación de experiencias web más interactivas y dinámicas.

Para adentrarse en el mundo del DOM, es esencial comenzar con la comprensión de cómo se estructura un documento HTML. En esta oportunidad, exploraremos las relaciones jerárquicas entre los elementos HTML y cómo el DOM refleja esta estructura en forma de nodos interconectados. También abordaremos conceptos clave, como la selección de elementos mediante identificadores, clases o etiquetas, así como la manipulación de atributos y contenido.

Revisaremos algunos conceptos relevantes como: **Nodos**, que son los elementos básicos del DOM; los **NodeList**, que son una colección ordenada de nodos obtenidos por métodos como `querySelectorAll` o propiedades como `childNodes` de ciertos elementos; la **Selección de Elementos**, útil para interactuar con el DOM; la **Modificación de Contenido**, donde podrás generar cambios de texto, atributos o incluso agregar y eliminar elementos; los **Atributos**, que existen dentro de los elementos HTML y a los cuáles puedes acceder con JavaScript; y finalmente también hablaremos de los **Eventos**, acciones o situaciones específicas que ocurren en una página web.

Gracias a esto, aprenderás a dinamizar sus páginas web mediante la escritura de scripts de JavaScript que interactúan con el DOM, permitiendo cambios en tiempo real en la interfaz de usuario. Desde la actualización de texto hasta la modificación de estilos y la manipulación de eventos, la utilización eficiente del DOM se convertirá en una habilidad clave para desarrollar aplicaciones web interactivas y modernas.

Resultado de aprendizaje

El estudiante será capaz de:

RA1. Utiliza elementos básicos de programación en JavaScript en sitio web, de acuerdo criterios de usabilidad y accesibilidad y los requerimientos del proyecto.

RA2. Programa en JavaScript, considerando el control de elementos BOM (browser) y DOM (documento) para un sitio web bajo criterios de usabilidad, accesibilidad y estéticos.

Indicadores de logro:

IL1. Utiliza correctamente los elementos básicos de programación como variables, arrays, operadores (if - else) y bucles, a fin de dar solución a los requerimientos del proyecto.

IL2. Utiliza el lenguaje JavaScript para programar funciones que permiten reutilizar código de manera eficiente.

IL3. Aplica las funciones y aplicaciones del BOM y DOM en el proceso de desarrollo de un sitio web con JavaScript.

IL4. Utiliza JavaScript para seleccionar y controlar elementos del DOM de manera dinámica, a fin de crear un sitio web que responda a las acciones del usuario.

IL5. Manipula los elementos del DOM, a fin de mejorar la usabilidad, accesibilidad y estética del sitio web.

Conceptos relevantes

	Nodos	Selección de elementos	Modificación de contenido
	Atributos	Eventos	NodeList

Preguntas activadoras

- ¿Sabes cómo seleccionar elementos en el DOM utilizando JavaScript?
- ¿Conoces la diferencia entre innerText e innerHTML?
- ¿Te gustaría aprender a crear funciones que permitan modificar el aspecto de una página a los visitantes de un sitio?

Actividad

Descripción de la actividad

En esta actividad formativa, tendrás que manipular el DOM utilizando JavaScript. Podrás cambiar el color del texto y la familia de fuentes de un HTML base, para que los encabezados tengan un aspecto visual determinado. Junto con lo anterior, aprenderás a modificar textos internos, así como asignar rutas para las imágenes que se mostrarán en tu sitio, entre otras particularidades.

¿Qué es el DOM?

El DOM, o Document Object Model, representa la estructura de un documento HTML (o XML) como un árbol de objetos. En otras palabras, proporciona una representación estructurada y jerárquica de todos los elementos de una página web, permitiendo a los programadores acceder y manipular dinámicamente el contenido, la estructura y el estilo de un documento.

Al utilizar el DOM, los desarrolladores web pueden:

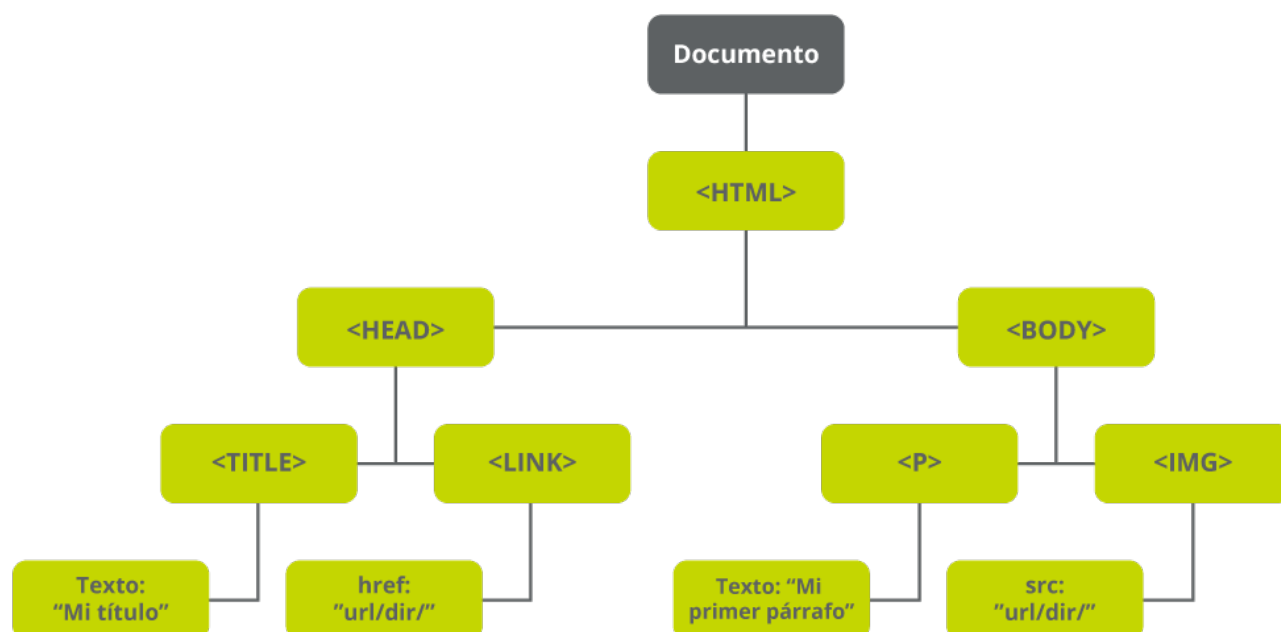
- **Acceder a Elementos HTML:** DOM proporciona métodos y propiedades que permiten a los desarrolladores acceder a elementos específicos en una página web. Esto facilita la interacción con el contenido, como la modificación del texto, la actualización de estilos o la manipulación de atributos.
- **Modificar Dinámicamente la Página:** Los scripts de JavaScript pueden utilizar el DOM para realizar cambios en la página después de que ha cargado inicialmente.
- **Responder a Eventos:** Posibilita la asignación de funciones a eventos específicos, como clics de ratón, pulsaciones de teclas o cambios en el estado de carga de la página. Esta capacidad permite crear interactividad y respuestas dinámicas a las acciones del usuario.
- **Manipular la Estructura del Documento:** Se pueden agregar, eliminar o reorganizar elementos en el DOM, lo que afecta directamente la estructura y el contenido de la página.

Árbol de objetos

Cuando hablamos de “árbol de elementos”, nos estamos refiriendo a la representación jerárquica de todos los elementos de un documento HTML como una estructura de objetos interconectados. Este árbol refleja la estructura y organización del documento, donde cada objeto en el árbol representa un elemento individual en la página web, como un elemento HTML, un atributo, o incluso un fragmento de texto.

Figura 1

Ejemplo de árbol de objetos de DOM



El DOM representa un documento como una estructura de árbol de nodos y relaciones, revisemos cada uno de estos componentes:

- **El nodo raíz**

Representa todo el documento y generalmente corresponde al objeto "documento".

- **Los nodos hijos**

Representan elementos HTML individuales y están conectados a la raíz. Cada elemento tiene nodos hijos que representan sus descendientes, como los elementos secundarios, atributos o texto contenido dentro de ese elemento. En la jerarquía del árbol del DOM, los nodos hijos se encuentran bajo otro nodo específico, conocido como su nodo padre.

- **Los nodos hermanos**

Son nodos que están al mismo nivel en la jerarquía del árbol del DOM, lo que significa que comparten el mismo nodo padre.

En el ejemplo de la **figura 1**, los elementos "<p>" e "" anidados bajo el elemento "<body>" serían hermanos.

La estructura de árbol facilita la navegación y manipulación de los elementos del documento. También permite entender cómo se relacionan los elementos entre sí y cómo los cambios en una parte del árbol pueden afectar a otras partes.

Links de interés

Si quieres profundizar en el DOM, puedes acceder a los siguientes recursos online:

- JavaScript.info (<https://es.javascript.info/basic-dom-node-properties>)
Explica las propiedades de los nodos DOM, su jerarquía y cómo se organizan en el árbol DOM las diferentes clases de nodos.
- El sitio web de Pablo Monteserín se enfoca en cómo utilizar el DOM y modificar el árbol de nodos en JavaScript. (<https://pablomonteserin.com/curso/javascript/dom/>)
- CódigoFacilito (<https://codigofacilito.com/articulos/javascript-dom-desarrollo-web>)
Tiene una publicación en su blog que profundiza en el DOM y explica cómo buscar elementos en el árbol DOM para modificarlos o para obtener los valores del elemento mismo.

Atributos

Un atributo es una propiedad adicional que se asigna a un elemento HTML para proporcionar información sobre ese elemento. Los atributos suelen tener la forma de pares clave-valor y se colocan en la etiqueta de apertura de un elemento HTML.

Con JavaScript, puedes acceder y modificar estos atributos para cambiar el comportamiento o la apariencia de un elemento. Algunos métodos útiles incluyen `getAttribute`` y `setAttribute``.

Por ejemplo, en la figura 2 se muestra un fragmento de código HTML donde, en la etiqueta `<a>` (hipervínculo), el atributo `"href"` se utiliza para especificar la URL a la que el enlace apunta. Así, `"href"` es el nombre del atributo y `"https://www.ejemplo.com"` es el valor asignado a ese atributo. Es decir, es el texto que los usuarios verán en la página web y sobre el cual se puede hacer clic para seguir el enlace.

Figura 2

Ejemplo de atributo

```
<a href="https://www.ejemplo.com">Enlace de Ejemplo</a>
```

Al usar DOM podemos acceder y modificar estos atributos. Por ejemplo, puedes cambiar dinámicamente el valor de un atributo para actualizar el comportamiento de un elemento en respuesta a ciertos eventos o condiciones.

Selección de elementos en el DOM

La selección de elementos en un documento web está íntimamente ligado a cómo los definimos y estructuramos en la página HTML.

Seleccionar un elemento con un id único es diferente de seleccionar un grupo de elementos con una class compartida, ya que al asignar identificadores (id) y clases (class) a los elementos HTML, se establecen maneras específicas de referenciar y agrupar estos elementos. Por ello, antes de revisar los diferentes métodos que existen para seleccionar elementos, haremos un breve repaso sobre cómo asignar identificadores a los tags de HTML.

Identificadores únicos: ID

En HTML, **id** es un atributo utilizado para identificar de manera única un elemento dentro de un documento. Cada elemento puede tener, a lo sumo, un atributo id, y este identificador no puede repetirse en el documento. Esto permite referenciar específicamente un elemento individual desde scripts de JavaScript, estilos CSS o enlaces internos.

Figura 3

Ejemplo de ID

```
<p id="miParrafo">Este es un párrafo con un ID específico.</p>
```

Identificadores masivos: class

Class es un atributo que se utiliza para asignar una o varias clases a un elemento. Las clases son útiles para aplicar estilos a un conjunto de elementos de manera consistente o para seleccionar y manipular grupos de elementos mediante JavaScript. Es importante recordar que, al trabajar con varios elementos, para acceder a ellos de forma individual es necesario recorrerlos como un bucle o array, utilizando **for** o **while**.

Figura 4

Ejemplo class

```
<p class="estilo-destacado">Este es un párrafo con una clase de estilo.</p>
```

Seleccionado elementos con Javascript

Una vez que hemos asignado identificadores o clases a nuestros elementos HTML, podemos utilizar métodos de selección en JavaScript.

Los métodos para seleccionar elementos varían dependiendo si vamos a trabajar con elementos individuales, masivos o directamente con tags de HTML. Por ello tenemos los siguientes métodos a nuestra disposición:

getElementById

Busca los elementos usando el ID asignado. Como el id de un elemento debe ser único por página web, usar el id es una forma perfecta de seleccionar un elemento específico de la jerarquía DOM, incluso si hay varios elementos similares o idénticos.

Figura 5

Sintaxis de búsqueda por ID

```
let elemento = document.getElementById("idDelElemento");
```

getElementsByClassName

Busca los elementos usando la clase asignada. Esto trabaja con un arreglo de objetos que tienen esta característica, por lo que debes recorrerlo usando **for** o **forEach**.

Figura 6

Sintaxis de búsqueda por Class

```
let elementos = document.getElementsByClassName("nombreDeClase");
```

getElementsByTagName

Busca los elementos usando el tag usado en html. Al igual que el anterior, al trabajar con múltiples elementos, es necesario recorrerlo como un arreglo de objetos usando **for** o **forEach**

Figura 7

Sintaxis de búsqueda por tag

```
let elementos = document.getElementsByTagName("nombreDeEtiqueta");
```

Ejemplos de uso de la búsqueda

Búsqueda por ID

En este caso creamos un **H1** con el **ID** “miTitulo”. Luego, en el javascript, creamos la variable “**titulo**” y le asignamos lo que encuentre al usar **getElementById**.

La función `getElementById` busca en todo el documento HTML un elemento que tenga un `id` que coincida con el valor proporcionado, que en este caso es `"miTitulo"`. Cuando lo encuentra, devuelve una referencia a ese elemento, que luego se almacena en la variable `titulo`. Después de ejecutar este código, la variable `titulo` ahora apunta al elemento `<h1>` en el DOM. Esto significa que puedes hacer cosas como cambiar el texto del encabezado.

Figura 8

Ejemplo de búsqueda por ID

```
<h1 id="miTitulo">¡Hola, Mundo!</h1>

<script>
  // Obtener el elemento con id "miTitulo"
  let titulo = document.getElementById("miTitulo");

</script>
```

Búsqueda por class

Veamos un ejemplo sobre cómo buscar elementos por su clase CSS y luego, cómo iterar sobre estos elementos para modificar su contenido.

En la figura 9 se muestra un fragmento de código HTML y JavaScript. La parte HTML indica que hay tres párrafos (**<p>**) en la página, dos de los cuales tienen la clase **"resaltado"**. En el script JavaScript, **getElementsByName("resaltado")** se utiliza para seleccionar todos los elementos que tienen la clase "resaltado" y obtener una colección de todos los elementos que coinciden, que en este caso son los elementos **<p>** con la clase "resaltado". Finalmente, se utiliza un bucle **for** para iterar sobre la colección y cambiar el contenido de cada elemento resaltado. De este modo, en cada iteración, el contenido de cada elemento con la clase "resaltado" se modificará añadiendo la palabra Actualizado al final del contenido existente. El segundo párrafo permanecerá sin cambios, ya que no tiene la clase "resaltado".

Figura 9

Ejemplo de búsqueda por Class

```
<p class="resaltado">Este párrafo está resaltado.</p>
<p>Este párrafo no está resaltado.</p>
<p class="resaltado">Este párrafo también está resaltado.</p>

<script>
  // Obtener todos los elementos con la clase "resaltado"
  let elementosResaltados = document.getElementsByClassName("resaltado");

  // Iterar sobre la colección y cambiar el contenido de cada elemento
  for (let i = 0; i < elementosResaltados.length; i++) {
    elementosResaltados[i].innerHTML += " (Actualizado)";
  }
</script>
```

Búsqueda por tag

En el ejemplo de la figura 10, hay una lista no ordenada () con tres elementos de lista (). Cada contiene un texto que es "Elemento 1", "Elemento 2" y "Elemento 3" respectivamente.

En el script, se utiliza **getElementsByTagName("li")** para seleccionar todos los elementos del DOM que coinciden con el nombre de etiqueta especificado, en este caso "li" y obtener una colección de todos los elementos de lista en la página.

Finalmente, se utiliza un bucle **for** para iterar sobre cada elemento en la colección elementosLista y cambiar el contenido de cada uno de ellos a "Nuevo elemento" seguido del número del índice actual más uno. Es decir, se reemplaza "Elemento 1", "Elemento 2", y "Elemento 3" por "Nuevo elemento 1", "Nuevo elemento 2" y "Nuevo elemento 3".

Figura 10

Ejemplo de búsqueda por tag

```
<ul>
  <li>Elemento 1</li>
  <li>Elemento 2</li>
  <li>Elemento 3</li>
</ul>

<script>
  // Obtener todos los elementos "li" dentro de la lista
  var elementosLista = document.getElementsByTagName("li");

  // Iterar sobre la colección y cambiar el contenido de cada elemento
  for (var i = 0; i < elementosLista.length; i++) {
    elementosLista[i].innerHTML = "Nuevo elemento " + (i + 1);
  }
</script>
```

Modificar elementos

Como te habrás dado cuenta en las figuras 8 y 9, existen formas de alterar el contenido de los elementos de un documento WEB. En la práctica esto se traduce en que puedes manipular textos, imágenes, agregar o eliminar tags y mucho más.

Exploremos qué métodos permiten esta manipulación:

element.innerHTML()

Se usa para obtener o establecer el contenido HTML de un elemento. Esto permite cambiar todo el contenido interno de un elemento, incluyendo texto y elementos HTML anidados. En el ejemplo de la figura 13, el script obtiene el elemento **div** con el **id** “miDiv”. Muestra lo que contiene dentro del HTML (en este caso, el texto “Contenido inicial”), y finalmente lo modifica por el texto “Nuevo contenido”.

Figura 13

Ejemplo de innerHTML

```
<div id="miDiv">Contenido inicial</div>

<script>
  let miElemento = document.getElementById("miDiv");
  console.log(miElemento.innerHTML); // Muestra: Contenido inicial

  miElemento.innerHTML = "Nuevo contenido";
</script>
```

element.innerText()

Se utiliza para obtener o establecer el contenido de texto de un elemento, excluyendo cualquier etiqueta HTML presente. Es útil para cambiar solo el texto, sin afectar a ninguna estructura HTML que pueda estar presente.

En el ejemplo de la figura 14, el script obtiene el elemento **div** con el **id** “miDiv” y muestra el texto contenido dentro del HTML, excluyendo cualquier tag de html que pudiera haber dentro del elemento. Por tanto, el contenido visible del div ya no será "Contenido con HTML", sino que "Nuevo texto".

Figura 14

Ejemplo de innerText

```
<div id="miDiv">Contenido <span>con HTML</span></div>

<script>
  let miElemento = document.getElementById("miDiv");
  console.log(miElemento.innerText); // Muestra: Contenido con HTML

  miElemento.innerText = "Nuevo texto";
</script>
```

element.getAttribute()

Se utiliza para obtener el valor de un atributo específico de un elemento.

En el ejemplo de la figura 15, el script obtiene el contenido del link con el **ID** “miEnlace” y lo guarda en la variable **enlace**. Luego crea la variable **hrefValor** y le asigna el valor del atributo **href** del link que está contenido en la variable **enlace**. Finalmente, el script muestra el valor de la **variable hrefValor** en la consola del navegador usando **console.log**.

Figura 15

Ejemplo `getAttribute`

```
<a href="https://www.ejemplo.com" id="miEnlace">Enlace de ejemplo</a>

<script>
  let enlace = document.getElementById("miEnlace");
  let hrefValor = enlace.getAttribute("href");
  console.log(hrefValor); // Muestra: https://www.ejemplo.com
</script>
```

`element.setAttribute(atributo,valor)`

Se utiliza para establecer el valor de un atributo específico en un elemento.

A modo de ejemplo, en la figura 16 el script obtiene el elemento con el ID “**miDiv**” y lo asigna a la variable **miElemento**. Luego, le agrega el atributo **class** con el valor “**nuevaClase**”. Por tanto, después de ejecutar este script, el `<div>` tendría una clase CSS llamada ``nuevaClase``, lo que podría cambiar su estilo si en la hoja de estilos CSS hay definiciones para `.nuevaClase`.

Figura 16

Ejemplo `setAttribute`

```
<div id="miDiv">Contenido inicial</div>

<script>
  let miElemento = document.getElementById("miDiv");
  miElemento.setAttribute("class", "nuevaClase");
</script>
```

element.style()

Se utiliza para acceder a los estilos en línea de un elemento y modificarlos. Puedes cambiar propiedades como el color, tamaño de fuente, posición, etc.

En el ejemplo de la figura 17, el script obtiene el elemento con el ID “miDiv” y lo asigna a la variable **miElemento**. Luego, accede a la propiedad `style` del elemento seleccionado, específicamente a la propiedad `color` y modifica el color de la fuente usando **style.color**, asignándole el color azul. También establece el tamaño de la fuente en 18px usando **style.fontSize**.

Después de ejecutar este script, el texto "Texto de ejemplo" dentro del `

` aparecerá en color azul y con un tamaño de fuente de 18 píxeles.

Figura 17

Ejemplo style

```
<div id="miDiv">Texto de ejemplo</div>

<script>
  let miElemento = document.getElementById("miDiv");
  miElemento.style.color = "blue";
  miElemento.style.fontSize = "18px";
</script>
```

Operaciones sobre conjuntos de elementos

Cuando deseas manipular o acceder a múltiples elementos del DOM que tienen algo en común, ya sea porque comparten la misma clase (en el caso de **getElementsByClassName**) o coinciden con un selector CSS particular (en el caso de **querySelectorAll**), puedes iterar a través de ellos utilizando un bucle **for** o **forEach** y realizar operaciones en cada elemento individual como, por ejemplo, cambiar estilos, modificar contenidos o establecer atributos.

Figura 18

Ejemplo con `querySelectorAll` y `forEach`

```
var elementos = document.querySelectorAll(".miClase");

elementos.forEach(function(elemento) {
  // Hacer algo con cada elemento
  elemento.style.color = "blue";
});
```

Recuerda que **querySelectorAll** devuelve una **NodeList**, y si necesitas realizar operaciones de array más avanzadas, puedes convertirla a un **array** usando **Array.from** o usando el operador de propagación (**...NodeList**).

Figura 19

Ejemplo de conversión de `NodeList` a `Array`

```
let elementosArray1 = Array.from(document.querySelectorAll(".miClase"));

// o usando el operador de propagación
let elementosArray2 = [...document.querySelectorAll(".miClase")];
```

Cierre de la semana

Al final de esta semana has aprendido que el DOM es la representación que utiliza el navegador de la estructura de un documento HTML, visualizada como un árbol de nodos jerárquicos que facilita la interacción con los elementos de la página.

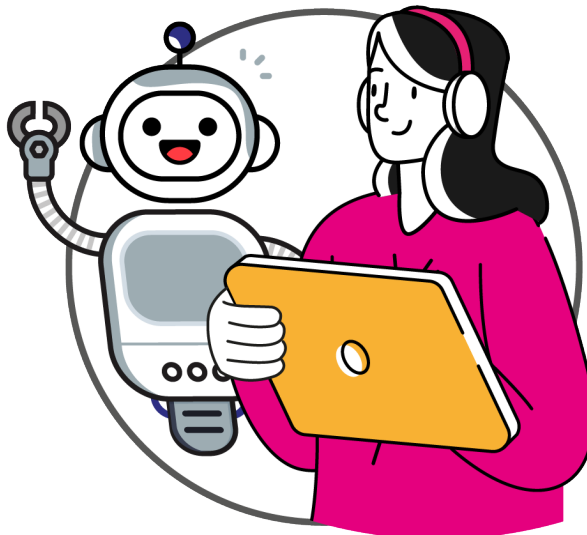
Conociste distintas maneras de seleccionar elementos dentro del documento utilizando métodos como `getElementById` para seleccionar elementos individuales, `getElementsByClassName` y

`getElementsByTagName` para grupos de elementos con la misma clase o etiqueta.

También pudiste comprender que es distinto trabajar con elementos únicos que, con múltiples elementos a la vez, y que para el segundo caso necesitas procesarlos usando `for` o `foreach`.

Finalmente, aprendiste cómo cambiar dinámicamente el contenido de una página web manipulando propiedades y métodos como `innerHTML` y `innerText` para modificar el texto y el HTML interno, `getAttribute` y `setAttribute` para trabajar con atributos de los elementos, y `style` para alterar los estilos en línea de los elementos.

Ahora estás listo para comenzar a aplicar todo lo aprendido. ¡Adelante!



Referencias

- Ramírez, L. (s.f.). *JavaScript, 5 cosas que hay que saber*. <http://js.luisfel.cl>
- Mozilla web docs. (s.f.). *Guía de JavaScript*.
<https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Introducci%C3%B3n>

This image shows a full page of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page, typical of notebook paper. There are no margins, text, or other markings on the page.



DuocUC[®] ONLINE

Facilitador disciplinar: Claudio Navarro

Asesora par: Paola Véliz

Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.